

Recherche et planification : présentation du fascicule

Ce fascicule traite d'une famille de problèmes que l'on rencontre bien avant l'apprentissage automatique : trouver une suite d'actions qui mène d'une situation de départ à une situation désirée.

Deux questions le structurent. Comment décrire un problème sous une forme que l'on puisse traiter automatiquement ? Et comment explorer l'ensemble des solutions possibles sans en parcourir la totalité ?

Les algorithmes présentés ici ne font appel à aucune donnée statistique. Ils sont déterministes et fournissent des garanties précises sur leur résultat, à condition que le problème soit correctement modélisé.

Ce fascicule suit le fascicule 0 dans l'ordre de lecture, mais il peut être étudié avant le fascicule 2, qui formalise les mêmes questions en logique.

Qu'est-ce qu'un problème de recherche ?

Un problème de recherche est décrit par cinq éléments : un état initial, un ensemble d'actions applicables, un modèle de transition, un test de but et une fonction de cout.

Résoudre le problème consiste à trouver une suite d'actions qui transforme l'état initial en un état satisfaisant le test de but, en minimisant si possible le cout total.

Le mot état désigne ici une situation complète du problème, et non une étape d'un calcul. Cette distinction est importante : un état contient toute l'information nécessaire pour décider de la suite.

La difficulté vient de la taille de l'ensemble des états. Même pour des problèmes modestes, cet ensemble dépasse largement ce que l'on peut énumérer en pratique.

Formaliser un problème : les cinq composantes

L'état initial. La situation de départ, décrite de façon complète. Pour un problème de navigation, la ville de départ suffit ; pour un jeu, il faut la position de toutes les pièces.

Les actions. L'ensemble des transformations applicables à un état donné. Chaque action possède une précondition, éventuellement vide, qui détermine les états où elle s'applique.

Le modèle de transition. La description précise de l'état obtenu après application d'une action. Ce modèle doit être déterministe dans le cadre de ce fascicule : une action donnée sur un état donné produit toujours le même résultat.

Le test de but. Une condition qui caractérise les états acceptables. Elle peut définir un état unique ou un ensemble d'états, ce qui change la difficulté du problème.

La fonction de cout. Le cout d'un chemin est la somme des couts de ses actions. Lorsque tous les couts valent un, la recherche du cout minimal revient à la recherche du chemin le plus court en nombre d'actions.

Espace d'états et arbre de recherche

L'espace d'états est l'ensemble de tous les états atteignables depuis l'état initial. Il forme un graphe orienté dont les noeuds sont les états et les arcs les actions.

L'arbre de recherche est une autre structure : c'est l'arbre déplié par un algorithme au fil de son exploration. Un même état peut y apparaître plusieurs fois, atteint par des chemins différents.

Confondre les deux conduit à deux erreurs classiques : croire qu'un algorithme explore tout l'espace d'états alors qu'il n'en voit qu'une partie, et oublier de détecter les chemins qui reviennent sur un état déjà visité.

La taille de l'arbre de recherche dépend du facteur de branchement, noté b , c'est-à-dire du nombre moyen d'actions applicables, et de la profondeur atteinte, notée d .

Exemple : navigation sur une grille

Une grille rectangulaire comporte des cases libres et des obstacles. L'état initial est la case de départ, l'état but la case d'arrivée. Les actions sont les quatre déplacements vers une case voisine libre.

Le cout de chaque action vaut un, sauf si l'on souhaite favoriser certains déplacements. Le test de but compare la position courante à la position d'arrivée.

Ce problème est le plus simple que l'on puisse formuler, et il sert de banc d'essai : on peut y dessiner l'arbre de recherche et compter les états développés par chaque algorithme.

Sa simplicité est trompeuse. Dès que la grille grandit, le nombre de chemins possibles croît de façon exponentielle, et les différences entre algorithmes deviennent spectaculaires.

Exemple : le taquin

Le taquin est une grille de neuf cases dont huit portent un numéro et une est vide. Une action consiste à glisser dans la case vide une tuile adjacente.

Un état est une permutation des huit tuiles, soit plus de trois cent mille configurations. Seule la moitié de ces configurations sont atteignables depuis une configuration donnée.

Le test de but impose un ordre précis des tuiles. Trouver une solution proche de l'optimum demande un algorithme informé : les méthodes non informées y développent des centaines de milliers d'états.

Le taquin est l'exemple historique des recherches sur les heuristiques. La distance de Manhattan, qui additionne les distances de chaque tuile à sa position finale, en est l'heuristique classique.

Exemple : planification de tournée

Un technicien doit visiter plusieurs sites dans la journée. Chaque site a une durée d'intervention et une plage horaire. L'état décrit les sites déjà visités et l'heure courante.

Contrairement aux exemples précédents, le cout des actions varie et les contraintes temporelles rendent certains enchainements impossibles. Le test de but exige que tous les sites soient visités.

Ce problème appartient à la famille des problèmes d'ordonnancement. Sa résolution exacte devient rapidement impossible, et l'on se contente souvent d'une solution bonne sans être optimale.

La modélisation reste la même : états, actions, transitions, but, cout. Ce qui change est la taille de l'espace d'états et la nécessité d'accepter une solution approchée.

Comment évaluer un algorithme de recherche

Complétude. L'algorithme trouve-t-il une solution lorsqu'il en existe une ? Un algorithme incomplet peut échouer sur des problèmes qui ont une solution.

Optimalité. La solution trouvée est-elle de cout minimal parmi toutes les solutions ? Cette propriété est plus forte que la complétude et ne l'implique pas.

Complexité en temps. Combien d'états l'algorithme développe-t-il avant de trouver la solution ? On l'exprime en fonction du facteur de branchement et de la profondeur.

Complexité en espace. Combien d'états l'algorithme doit-il garder en mémoire ? Cette mesure est déterminante en pratique : c'est presque toujours la mémoire, et non le temps, qui limite la taille des problèmes traitables.

Mesurer la difficulté : b, d et m

Trois quantités suffisent à décrire la difficulté d'un problème de recherche. Le facteur de branchement b est le nombre maximal d'actions applicables dans un état.

La profondeur de la solution la plus proche, notée d , mesure la longueur du chemin que l'algorithme doit trouver. La profondeur maximale de l'espace, notée m , borne la longueur des chemins explorés.

Lorsque toutes les actions ont le même coût, le nombre d'états à la profondeur d vaut environ b^d . Pour b égal à dix et d égal à dix, cela représente dix milliards de noeuds.

Cette croissance explique pourquoi les algorithmes non informés ne suffisent pas, et pourquoi l'on cherche à utiliser une connaissance du domaine pour diriger l'exploration.

Recherche en largeur

La recherche en largeur développe d'abord tous les états à une profondeur donnée, puis passe à la profondeur suivante. Elle utilise une file d'attente dans laquelle les états sont ajoutés à la fin et retirés au début.

Elle est complète dès que le facteur de branchement est fini : si une solution existe, elle sera atteinte. Elle est optimale lorsque toutes les actions ont le même coût.

Sa complexité en temps est de l'ordre de b^d . Sa complexité en espace est du même ordre, car il faut conserver tous les états du niveau courant et ceux du niveau suivant.

C'est là sa faiblesse : la mémoire nécessaire devient prohibitive bien avant le temps de calcul. Pour un facteur de branchement de dix et une profondeur de dix, elle doit stocker des milliards de nœuds.

Recherche en profondeur

La recherche en profondeur suit une branche jusqu'à son extrémité avant de revenir en arrière. Elle utilise une pile : les états sont ajoutés et retirés au même bout.

Sa consommation mémoire est bien meilleure que celle de la recherche en largeur : elle ne stocke que les états du chemin courant et leurs frères non explorés, soit de l'ordre de b multiplié par m .

En revanche, elle n'est ni complète ni optimale. Elle peut s'engager dans une branche infinie et ne jamais revenir, et la première solution trouvée n'est pas nécessairement la plus courte.

La recherche en profondeur reste utile lorsque l'on sait que les solutions sont nombreuses et que l'on se contente d'en trouver une, quelle qu'elle soit.

Profondeur limitée et approfondissement itératif

La recherche à profondeur limitée fixe une profondeur maximale l et traite les états à cette profondeur comme des culs-de-sac. Elle rend la recherche en profondeur complète à condition de choisir l supérieur ou égal à d .

Le choix de l est délicat : trop petit, on manque la solution ; trop grand, on perd l'avantage mémoire de la recherche en profondeur.

L'approfondissement itératif supprime le problème du choix. On lance la recherche à profondeur limitée avec l égal à un, puis deux, puis trois, jusqu'à trouver une solution.

Cette répétition semble couteuse, mais elle ne l'est pas : la majorité des noeuds se trouvent aux profondeurs basses, et le travail répété représente une fraction constante du travail total.

L'algorithme est complet et optimal pour des couts uniformes, avec une consommation mémoire linéaire.

Recherche de cout uniforme

Lorsque les actions ont des couts différents, explorer par niveaux n'a plus de sens : une action peut coûter un comme cent. La recherche de cout uniforme développe toujours l'état dont le cout accumulé depuis le départ est le plus faible.

Cet algorithme est complet et optimal dès que les couts sont positifs. Il se ramène à la recherche en largeur lorsque tous les couts valent un.

Il souffre du même défaut que la recherche en largeur : il conserve tous les états générés, et sa complexité en espace reste exponentielle dans le pire cas.

Sa structure est celle d'une file de priorité ordonnée par cout accumulé. L'algorithme A étoile, présenté plus loin, ajoute à ce critère une estimation du cout restant.

Recherche bidirectionnelle

Principe. On explore simultanément depuis l'état initial et depuis l'état but, et l'on s'arrête lorsque les deux explorations se rencontrent.

L'intérêt est arithmétique : si deux recherches atteignent chacune la profondeur d sur deux, leur réunion couvre la profondeur d avec un travail très inférieur à celui d'une exploration complète depuis le départ.

La difficulté est pratique. Il faut savoir énumérer les prédécesseurs d'un état, ce qui n'est pas toujours naturel, et décider quand les deux fronts se sont effectivement rejoints.

La méthode s'applique bien aux problèmes où l'action est réversible, comme la navigation, moins bien à ceux où le but est défini par une condition abstraite.

Recherche informée : l'idée d'heuristique

Une fonction heuristique associe à chaque état une estimation du cout restant jusqu'au but. Elle ne dit rien sur le chemin, seulement sur le cout à venir.

Une heuristique s'obtient de plusieurs façons : en résolvant une version simplifiée du problème, en utilisant une distance géométrique, ou en combinant plusieurs estimations.

Une bonne heuristique n'est pas nécessairement exacte. Elle doit être informative, c'est-à-dire distinguer les états proches du but de ceux qui en sont loin, tout en restant prudente sur le cout.

La qualité de l'heuristique détermine directement le nombre d'états que l'algorithme doit développer. C'est le principal levier d'amélioration des algorithmes de recherche.

Recherche gloutonne

La recherche gloutonne développe toujours l'état dont l'estimation du cout restant est la plus faible. Elle ignore complètement le cout déjà payé pour l'atteindre.

Elle est rapide et peu gourmande en mémoire, mais ni complète ni optimale. Comme la recherche en profondeur, elle peut s'enfermer dans une branche sans issue.

Sa faiblesse est facile à illustrer : entre un état proche du but atteint au prix d'un long détour et un état plus éloigné mais atteint directement, elle choisit le premier.

L'heuristique doit donc être conçue pour ce critère. Une heuristique très informative mais mal calibrée peut conduire la recherche gloutonne vers un chemin absurde.

L'algorithme A étoile

A étoile classe les états par la somme de deux termes : le cout déjà payé pour atteindre l'état depuis le départ, et l'estimation du cout restant jusqu'au but.

Cette somme est une estimation du cout total du meilleur chemin passant par l'état considéré. L'algorithme développe toujours l'état dont cette estimation est la plus faible.

A étoile combine les avantages des deux approches précédentes : il tient compte du cout réel comme la recherche de cout uniforme, et il est dirigé vers le but comme la recherche gloutonne.

Sous une condition précise sur l'heuristique, A étoile est à la fois complet et optimal. C'est cette condition qui fait de lui l'algorithme de référence pour la recherche de chemin.

Heuristique admissible et optimalité de A étoile

Une heuristique est dite admissible si elle ne surestime jamais le cout réel restant jusqu'au but. Elle peut sous-estimer autant qu'elle veut, tant qu'elle reste positive.

Si l'heuristique est admissible, A étoile est optimal. L'argument est simple : une solution sous-optimale aurait un cout total supérieur à celui de la solution optimale, et l'estimation d'un état situé sur le chemin optimal reste toujours en dessous.

Une seconde condition, plus forte, est la consistence : l'estimation d'un état ne doit pas dépasser le cout d'une action plus l'estimation de l'état suivant. Une heuristique consistante est admissible, et elle garantit qu'aucun état n'est développé deux fois.

En pratique, la distance de Manhattan pour le taquin et la distance à vol d'oiseau pour la navigation sont admissibles. Multiplier une heuristique admissible par un facteur supérieur à un détruit cette propriété.

Construire une heuristique

Méthode des problèmes relaxés. On supprime une contrainte du problème : le cout de la solution du problème simplifié est une estimation optimiste du cout réel. C'est la méthode la plus systématique.

Pour le taquin, autoriser une tuile à se déplacer vers n'importe quelle case donne la distance de Manhattan. Autoriser une tuile à se déplacer vers une case voisine occupée donne une autre estimation, plus faible.

Combiner des heuristiques. Le maximum de plusieurs heuristiques admissibles reste admissible, et il est souvent plus informatif que chacune prise séparément.

Bases de données de motifs. On précalcule le cout exact de sous-problèmes récurrents, puis on utilise ces valeurs comme estimation. Cette technique a permis de résoudre le taquin à quinze cases de façon optimale.

Mesurer la qualité d'une heuristique

Le facteur de branchement effectif mesure le nombre de successeurs réellement développés par A étoile, comparé à ce qu'il faudrait pour atteindre la même profondeur avec une heuristique parfaite.

Si une heuristique réduit le facteur de branchement effectif de dix à trois, le gain est considérable : à profondeur dix, le nombre d'états passe de milliards à quelques dizaines de milliers.

Deux heuristiques peuvent être comparées en comparant leur facteur de branchement effectif sur la même famille de problèmes. C'est une mesure empirique, mais elle guide efficacement le travail de conception.

Attention toutefois à ne pas comparer des heuristiques sur des instances différentes : la conclusion serait aussi peu fiable que celle d'un benchmark mal construit.

Recherche locale : la montée de colline

La recherche locale abandonne l'idée de chemin. Elle maintient un état courant et passe à un voisin meilleur tant qu'il en existe un.

La montée de colline est immédiate à implémenter et consomme très peu de mémoire, puisqu'elle ne conserve qu'un état. Elle est utilisée pour les problèmes où seule importe la solution finale, pas le chemin.

Elle souffre de trois blocages classiques : les maxima locaux, les plateaux où tous les voisins ont la même valeur, et les crêtes étroites que l'on ne peut suivre qu'en dégradant temporairement la solution.

Ces blocages ne sont pas des cas pathologiques rares : sur de nombreux problèmes combinatoires, ils constituent la difficulté principale.

Recherche locale : recuit simulé et faisceaux

Le recuit simulé accepte parfois une dégradation, avec une probabilité qui diminue au fil du temps. Cette tolérance temporaire permet de sortir des maxima locaux.

Le nom vient de la métallurgie, où l'on chauffe puis refroidit lentement un métal pour lui faire atteindre une structure de faible énergie. Le parallèle est formel : la température contrôle la probabilité d'accepter une dégradation.

La recherche en faisceaux conserve au contraire un petit nombre d'états courants à chaque étape, les meilleurs. Elle explore donc plusieurs pistes en parallèle, au prix d'une mémoire plus grande.

Ces méthodes ne fournissent aucune garantie d'optimalité. Leur intérêt est de produire rapidement une bonne solution sur des problèmes où la résolution exacte est hors de portée.

Vers la planification automatique

La planification automatique applique la recherche à des problèmes décrits dans un langage formel. On ne décrit plus les états un par un, mais les prédicats qui les caractérisent et les actions qui les transforment.

Une action est décrite par ses préconditions et ses effets : ce qui doit être vrai pour l'appliquer, et ce qui devient vrai ou faux après. Ce formalisme permet de décrire des domaines entiers sans énumérer les états.

La difficulté principale est que l'espace d'états est décrit de façon implicite. Les algorithmes doivent le parcourir sans le construire entièrement, ce qui exige des techniques d'exploration dirigée.

Le fascicule 2 présente l'autre voie de formalisation : décrire le problème par des formules logiques et chercher une affectation qui les satisfait.

Ce qu'il faut retenir de ce fascicule

Un problème de recherche se décrit toujours par les cinq mêmes composantes : état initial, actions, transition, test de but et cout. La qualité de la modélisation détermine la difficulté réelle du problème.

Les algorithmes non informés sont complets ou optimaux, mais leur complexité exponentielle limite leur usage aux problèmes de petite taille. C'est la mémoire qui est le plus souvent contraignante.

L'algorithme A étoile est optimal dès que l'heuristique est admissible, ce qui donne un critère vérifiable pour la concevoir. La méthode des problèmes relaxés fournit une façon systématique d'en construire.

Enfin, lorsque seule la solution compte, la recherche locale offre un compromis intéressant : aucune garantie, mais des solutions exploitables à très grande échelle.

Exercices du fascicule 1

Exercice 1. Modéliser le problème du loup, de la chèvre et du chou sous forme d'espace d'états. Donner l'état initial, les actions, le test de but, puis dessiner le graphe complet des états atteignables.

Exercice 2. Comparer la recherche en largeur, la recherche en profondeur et l'approfondissement itératif sur le taquin à huit cases, en comptant les états développés pour trois configurations initiales.

Exercice 3. Montrer que la distance de Manhattan est admissible pour le taquin, puis comparer le facteur de branchement effectif de cette heuristique avec celui obtenu par le nombre de tuiles mal placées.

Exercice 4. Proposer une heuristique admissible pour le problème de tournée de la page précédente, puis indiquer dans quel cas elle est la plus informative.